

```

import React, { useState } from 'react';
import { ChevronDown, Plus, Search, Leaf, Zap, PackageOpen, AlertCircle, Globe }

// Product Master Database Structure
const PRODUCT_DATABASE = {
  categories: {
    'ML': {
      name: 'Молочные',
      icon: '🥛',
      color: '#E8F0FE',
      subcategories: {
        'CR': { name: 'Сливки', icon: '🍦' },
        'YG': { name: 'Йогурт', icon: '🥄' },
        'CH': { name: 'Сыр', icon: '🧀' },
        'MK': { name: 'Молоко', icon: '🥛' }
      }
    },
    'VG': {
      name: 'Овощи',
      icon: '🥬',
      color: '#E6F9E6',
      subcategories: {
        'TM': { name: 'Томаты', icon: '🍅' },
        'PP': { name: 'Перец', icon: '🌶️' },
        'ON': { name: 'Лук', icon: '🧅' },
        'GB': { name: 'Зелень', icon: '🌿' }
      }
    },
    'MT': {
      name: 'Мясо & Рыба',
      icon: '🍖',
      color: '#FEE8E8',
      subcategories: {
        'CH': { name: 'Курица', icon: '🍗' },
        'BF': { name: 'Говядина', icon: '🥩' },
        'FH': { name: 'Рыба', icon: '🐟' },
        'SF': { name: 'Морепродукты', icon: '🦞' }
      }
    },
    'PA': {
      name: 'Панیر & Основы',
      icon: '🥙',
      color: '#FFF4E6',
      subcategories: {

```

```

        'OL': { name: 'Масла', icon: '🥛' },
        'BR': { name: 'Крупы', icon: '🌾' },
        'PS': { name: 'Паста', icon: '🍝' },
        'SL': { name: 'Соль & Специи', icon: '🧂' }
    }
}
};

```

// Sample Products with full Master data

```

const SAMPLE_PRODUCTS = [
    {
        id: 'ML-CR-001',
        category: 'ML',
        subcategory: 'CR',
        russianName: 'СЛИВКИ 33%',
        englishName: 'Cream 33%',
        originalName: 'Slagroom',
        baseUnit: 'ml',
        storageType: 'Refrigerated',
        shelfLife: 10,
        openShelfLife: 3,
        minimumStock: 500,
        defaultCurrency: 'EUR',
        averagePrice: 2.49,
        active: true,
        supplier: 'Albert Heijn'
    },
    {
        id: 'VG-TM-001',
        category: 'VG',
        subcategory: 'TM',
        russianName: 'Томаты',
        englishName: 'Tomatoes',
        originalName: 'Tomaten',
        baseUnit: 'kg',
        storageType: 'Room Temperature',
        shelfLife: 7,
        openShelfLife: 0,
        minimumStock: 1000,
        defaultCurrency: 'EUR',
        averagePrice: 3.99,
        active: true,
        supplier: 'Market'
    },
    {
        id: 'MT-CH-001',

```

```

        category: 'MT',
        subcategory: 'CH',
        russianName: 'Куриное филе',
        englishName: 'Chicken Fillet',
        originalName: 'Kipfilet',
        baseUnit: 'kg',
        storageType: 'Frozen',
        shelfLife: 180,
        openShelfLife: 2,
        minimumStock: 500,
        defaultCurrency: 'EUR',
        averagePrice: 12.99,
        active: true,
        supplier: 'Albert Heijn'
    },
    {
        id: 'PA-OL-001',
        category: 'PA',
        subcategory: 'OL',
        russianName: 'Оливковое масло',
        englishName: 'Olive Oil',
        originalName: 'Olijfolie',
        baseUnit: 'ml',
        storageType: 'Pantry',
        shelfLife: 730,
        openShelfLife: 365,
        minimumStock: 250,
        defaultCurrency: 'EUR',
        averagePrice: 8.99,
        active: true,
        supplier: 'Albert Heijn'
    }
];

// Inventory Items (separate from Product Master)
const SAMPLE_INVENTORY = [
    {
        inventoryId: 'INV-001',
        productId: 'ML-CR-001',
        quantity: 500,
        unit: 'ml',
        location: 'Refrigerator',
        purchaseDate: '2024-01-15',
        expirationDate: '2024-01-25',
        openDate: null,
        status: 'Fresh',
        notes: ''
    }
];

```

```

    },
    {
      inventoryId: 'INV-002',
      productId: 'VG-TM-001',
      quantity: 2000,
      unit: 'g',
      location: 'Prep',
      purchaseDate: '2024-01-18',
      expirationDate: '2024-01-25',
      openDate: null,
      status: 'Fresh',
      notes: 'From market'
    },
    {
      inventoryId: 'INV-003',
      productId: 'MT-CH-001',
      quantity: 1500,
      unit: 'g',
      location: 'Freezer',
      purchaseDate: '2024-01-10',
      expirationDate: '2025-07-10',
      openDate: null,
      status: 'Frozen',
      notes: ''
    }
  ]
};

// Architecture Visualization
const ArchitectureFlow = () => (
  <div className="space-y-3 mb-8">
    <div className="text-xs font-mono text-gray-400 uppercase tracking-wider mb-4">
      <div className="flex items-center gap-2 overflow-x-auto pb-2">
        ['Product Master', 'Inventory', 'Recipes', 'Menu', 'Shopping', 'Analytics']
        <React.Fragment key={i}>
          <div className="px-3 py-1.5 rounded bg-gradient-to-br from-blue-500 to-
            {stage}
          </div>
          {i < 5 && <div className="text-blue-500">→</div>}
        </React.Fragment>
      </div>
    </div>
  </div>
);

// Product Master Editor Component
const ProductMasterEditor = () => {
  const [selectedCategory, setSelectedCategory] = useState('ML');

```

```

const [searchTerm, setSearchTerm] = useState('');
const [viewMode, setViewMode] = useState('grid'); // grid or structure

const filteredProducts = SAMPLE_PRODUCTS.filter(p =>
  p.category === selectedCategory &&
  (p.russianName.toLowerCase().includes(searchTerm.toLowerCase()) ||
  p.id.toLowerCase().includes(searchTerm.toLowerCase()))
);

const category = PRODUCT_DATABASE.categories[selectedCategory];

return (
  <div className="space-y-6">
    {/* Search and Filter */}
    <div className="space-y-3">
      <div className="relative">
        <Search className="absolute left-3 top-3 w-4 h-4 text-gray-500" />
        <input
          type="text"
          placeholder="Поиск товара или Product ID..."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
          className="w-full pl-10 pr-4 py-2 bg-gray-900 border border-gray-700"
        />
      </div>

      {/* Category Tabs */}
      <div className="flex gap-2 overflow-x-auto pb-2">
        {Object.entries(PRODUCT_DATABASE.categories).map(([key, cat]) => (
          <button
            key={key}
            onClick={() => setSelectedCategory(key)}
            className={`px-3 py-1.5 rounded text-sm font-medium transition ${
              selectedCategory === key
                ? 'bg-blue-600 text-white'
                : 'bg-gray-800 text-gray-300 hover:bg-gray-700'
            }`}
          >
            {cat.icon} {cat.name}
          </button>
        ))}
      </div>

      {/* Products Display */}
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        {filteredProducts.map(product => (

```

```

<div
  key={product.id}
  className="p-4 bg-gradient-to-br from-gray-800 to-gray-900 border bor
>
  {/* Product Header */}
  <div className="flex items-start justify-between mb-3">
    <div className="flex-1">
      <div className="text-xs font-mono text-blue-400 mb-1">{product.id}
      <div className="font-semibold text-white">{product.russianName}</
      <div className="text-xs text-gray-400 mt-1">{product.englishName}
    </div>
    <span className="text-2xl">{category.icon}</span>
  </div>

  {/* Key Properties */}
  <div className="grid grid-cols-2 gap-2 text-xs mb-3 pb-3 border-t bor
    <div>
      <span className="text-gray-500">Base Unit:</span> <span className=
    </div>
    <div>
      <span className="text-gray-500">Storage:</span> <span className="
    </div>
    <div>
      <span className="text-gray-500">Shelf Life:</span> <span classNam
    </div>
    <div>
      <span className="text-gray-500">Open:</span> <span className="tex
    </div>
  </div>

  {/* Price and Status */}
  <div className="flex items-center justify-between">
    <div>
      <span className="text-xs text-gray-500">Avg Price: </span>
      <span className="text-white font-semibold">{product.averagePrice}
    </div>
    <div className="px-2 py-1 bg-green-900/30 border border-green-700 r
      Active
    </div>
  </div>
</div>
  )})
</div>

{filteredProducts.length === 0 && (
  <div className="text-center py-8 text-gray-500">
    Нет товаров в этой категории

```

```

        </div>
      )}
    </div>
  );
};

// Inventory System Viewer
const InventoryViewer = () => {
  const getStatusColor = (status) => {
    const colors = {
      'Fresh': 'bg-green-900/30 text-green-300 border-green-700',
      'Opened': 'bg-yellow-900/30 text-yellow-300 border-yellow-700',
      'Frozen': 'bg-blue-900/30 text-blue-300 border-blue-700',
      'Expiring Soon': 'bg-orange-900/30 text-orange-300 border-orange-700',
      'Expired': 'bg-red-900/30 text-red-300 border-red-700'
    };
    return colors[status] || colors['Fresh'];
  };

  const getLocationIcon = (location) => {
    const icons = {
      'Refrigerator': '🧊',
      'Freezer': '❄️',
      'Pantry': '📦',
      'Prep': '🔍',
      'Ready Meals': '🍲'
    };
    return icons[location] || '📦';
  };

  return (
    <div className="space-y-4">
      <div className="text-xs font-mono text-gray-400 uppercase tracking-wider mb-3">
        {SAMPLE_INVENTORY.map(item => {
          const product = SAMPLE_PRODUCTS.find(p => p.id === item.productId);
          const category = PRODUCT_DATABASE.categories[product.category];

          return (
            <div key={item.inventoryId} className="p-4 bg-gray-800/50 border border-gray-700">
              <div className="flex items-start justify-between mb-3">
                <div className="flex-1">
                  <div className="flex items-center gap-2 mb-1">
                    <span className="text-2xl">{category.icon}</span>
                  </div>
                  <div className="text-xs font-mono text-gray-500">{item.inventoryId}</div>
                  <div className="font-semibold text-white">{product.russianName}</div>
                </div>
              </div>
            </div>
          );
        })}
      </div>
    </div>
  );
};

```

```

        </div>
      </div>
    </div>
    <div className={`px-2 py-1 rounded text-xs font-medium border ${get
      {item.status}}
    </div>
  </div>

  <div className="grid grid-cols-2 gap-3 text-xs mb-2">
    <div>
      <span className="text-gray-500">Quantity: </span>
      <span className="text-white font-semibold">{item.quantity} {item.
    </div>
    <div>
      <span className="text-gray-500">Location: </span>
      <span className="text-white font-semibold">{getLocationIcon(item.
    </div>
    <div>
      <span className="text-gray-500">Purchase: </span>
      <span className="text-white font-semibold">{item.purchaseDate}</s
    </div>
    <div>
      <span className="text-gray-500">Expires: </span>
      <span className="text-white font-semibold">{item.expirationDate}<
    </div>
  </div>

  <div className="pt-2 border-t border-gray-700">
    <span className="text-xs text-gray-400">Product ID: <span className
  </div>
</div>
  );
  }}}
</div>
);
};

// Structure Visualization
const StructureVisualization = () => (
  <div className="space-y-4">
    <div className="text-xs font-mono text-gray-400 uppercase tracking-wider mb-4

    {['ML', 'VG', 'MT', 'PA'].map(catKey => {
      const category = PRODUCT_DATABASE.categories[catKey];
      return (
        <div key={catKey} className="p-4 bg-gray-800/50 border border-gray-700 ro
          <div className="font-semibold text-white mb-3 flex items-center gap-2">

```

```
<span className="text-2xl">{category.icon}</span>
    {category.name}
</div>

<div className="ml-6 space-y-2">
  {Object.entries(category.subcategories).map(([subKey, sub]) => (
    <div key={subKey} className="text-sm text-gray-300">
      <div className="flex items-center gap-2 mb-1">
        <span className="text-gray-600">→</span>
        <span className="text-xl">{sub.icon}</span>
        <span>{sub.name}</span>
      </div>

      <div className="ml-6 text-xs text-gray-500">
        {SAMPLE_PRODUCTS.filter(p => p.category === catKey && p.subcate
          <div key={p.id} className="flex items-center gap-2 text-blue-
            <span>→</span>
            <span className="font-mono">{p.id}</span>
            <span>{p.russianName}</span>
          </div>
        )}}
      </div>
    </div>
  )})
</div>
);
}}}
</div>
);

// Main App
export default function HomechefProductMaster() {
  const [activeTab, setActiveTab] = useState('products');

  return (
    <div className="min-h-screen bg-gradient-to-br from-gray-950 via-gray-900 to-
      {/* Header */>
    <div className="border-b border-gray-800 bg-black/40 backdrop-blur sticky t
      <div className="max-w-7xl mx-auto px-6 py-4">
        <div className="flex items-center justify-between">
          <div>
            <h1 className="text-2xl font-bold bg-gradient-to-r from-blue-400 to
              Домашний Шеф
            </h1>
            <p className="text-xs text-gray-500 mt-1">Product Master System v1.
          </div>
```

```

        <div className="text-xs text-gray-600">
            🇺🇸 Multi-Currency | 🌐 Multilingual
        </div>
    </div>
</div>
</div>

{/* Navigation */}
<div className="border-b border-gray-800 bg-black/20 backdrop-blur">
    <div className="max-w-7xl mx-auto px-6">
        <div className="flex gap-6">
            {[
                { id: 'products', label: 'Product Master', icon: '📦' },
                { id: 'inventory', label: 'Inventory', icon: '📊' },
                { id: 'structure', label: 'Structure', icon: '🏗️' },
                { id: 'specs', label: 'Specifications', icon: '⚙️' }
            ].map(tab => (
                <button
                    key={tab.id}
                    onClick={() => setActiveTab(tab.id)}
                    className={`px-4 py-3 text-sm font-medium border-b-2 transition $
                        activeTab === tab.id
                        ? 'border-blue-500 text-blue-400'
                        : 'border-transparent text-gray-400 hover:text-gray-300'
                    `}
                >
                    {tab.icon} {tab.label}
                </button>
            )))
        </div>
    </div>
</div>

{/* Content */}
<div className="max-w-7xl mx-auto px-6 py-8">
    {/* Architecture Overview - Always Visible */}
    <ArchitectureFlow />

    {/* Tab Content */}
    {activeTab === 'products' && (
        <div>
            <div className="mb-6">
                <h2 className="text-xl font-bold mb-2">Product Master Database</h2>
                <p className="text-sm text-gray-400">
                    Центральная база всех продуктов. Каждый товар имеет уникальный Pr
                </p>
            </div>

```

```

        <ProductMasterEditor />
    </div>
  )}

{activeTab === 'inventory' && (
  <div>
    <div className="mb-6">
      <h2 className="text-xl font-bold mb-2">Inventory System</h2>
      <p className="text-sm text-gray-400">
        Отдельная система остатков. Связана с Product Master через Produc
      </p>
    </div>
    <InventoryViewer />
  </div>
)}

{activeTab === 'structure' && (
  <div>
    <div className="mb-6">
      <h2 className="text-xl font-bold mb-2">Product Structure</h2>
      <p className="text-sm text-gray-400">
        Иерархия: Категория → Подкатегория → Товар
      </p>
    </div>
    <StructureVisualization />
  </div>
)}

{activeTab === 'specs' && (
  <div className="space-y-6">
    <div>
      <h2 className="text-xl font-bold mb-4">Product Master Specification

      <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
        {/* Product Fields */}
        <div className="p-6 bg-gray-800/50 border border-gray-700 rounded">
          <h3 className="font-semibold mb-3 text-blue-400">Product Master
          <div className="space-y-2 text-sm text-gray-300">
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
            <div className="flex gap-2"><span className="text-blue-400">•
          </div>
        </div>
      </div>
    </div>
  </div>
)}

```

```

        <div className="flex gap-2"><span className="text-blue-400">•
        <div className="flex gap-2"><span className="text-blue-400">•
        <div className="flex gap-2"><span className="text-blue-400">•
        <div className="flex gap-2"><span className="text-blue-400">•
    </div>
</div>

{/* Inventory Fields */}
<div className="p-6 bg-gray-800/50 border border-gray-700 rounded
    <h3 className="font-semibold mb-3 text-green-400">Inventory Fie
    <div className="space-y-2 text-sm text-gray-300">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
        <div className="flex gap-2"><span className="text-green-400">
    </div>
</div>
</div>

{/* Storage Locations */}
<div className="mt-6 p-6 bg-gray-800/50 border border-gray-700 roun
    <h3 className="font-semibold mb-3">Storage Locations</h3>
    <div className="grid grid-cols-2 md:grid-cols-5 gap-4">
        {[
            <img alt="Refrigerator icon" data-bbox="325 603 345 618"/> Refrigerator',
            <img alt="Freezer icon" data-bbox="395 603 415 618"/> Freezer',
            <img alt="Pantry icon" data-bbox="465 603 485 618"/> Pantry',
            <img alt="Prep icon" data-bbox="535 603 555 618"/> Prep',
            <img alt="Box icon" data-bbox="605 603 625 618"/>
        <div key={loc} className="text-sm text-gray-300 p-2 bg-gray-9
            {loc}
        </div>
        )]}
    </div>
</div>

{/* Statuses */}
<div className="mt-6 p-6 bg-gray-800/50 border border-gray-700 roun
    <h3 className="font-semibold mb-3">Inventory Statuses</h3>
    <div className="grid grid-cols-2 md:grid-cols-4 gap-3">
        ['Fresh', 'Opened', 'Frozen', 'Defrosting', 'Defrosted', 'Expi
        <div key={status} className="text-xs text-gray-300 p-2 bg-gra
            {status}
        </div>
        )]}
    </div>

```

```

        </div>
    </div>
</div>
    })
</div>

{/* Footer Info */}
<div className="border-t border-gray-800 bg-black/20 mt-12">
    <div className="max-w-7xl mx-auto px-6 py-6 text-xs text-gray-500 space-y"
        <p>🔑 Product ID Naming: [Category]-[SubCategory]-[Number] (e.g., ML-CF
        <p>🔗 All sections connected via Product ID. Inventory references Produ
        <p>🌐 Multi-Currency Support (EUR, RUB). Multi-Language (Russian, Engli
    </div>
</div>
</div>
);
}

```